CONCISE SYSTEM ARCHITECTURE & ENGINEERING BLUEPRINT • 4-PAGE EDITION

AuctionArena: Automated Multi-Franchise Auction Engine

Architecture, State Flows, Concurrency, Deadlocks & Manual Debugging Guide (Stack-Agnostic)

ORCHESTRATION	GAME MODES	CODING RULE	CORE FOCUS
Zero Human Host (100% Automated)	1. Play vs Bots • 2. Friends Multiplayer	100% Manual Coding (AI = Critic Only)	Concurrency, Deadlocks & State Flow

1. Core Concept, Automated Lifecycle & Two Game Modes

SECTION 01

AuctionArena is a live IPL-style multi-franchise player auction simulator designed to teach backend systems thinking through manual implementation. **There is no human "Host" actor.** The backend auction engine acts as the automated auctioneer: it initializes the player pool, automatically starts the auction the moment team slots are ready, presents lots sequentially, enforces server-side countdown timers, and settles sales atomically. Every pattern you build here transfers directly to commercial systems like a *used-car dealership platform* (preventing double-sale of a unique VIN vehicle, managing buyer credit limits, and handling simultaneous reservations).

Operating Mode	User Setup & Room Creation Flow	Automated Engine Start Trigger
Mode 1: Play with Bots (Solo Arena)	User selects **Total Number of Teams** (e.g., 4, 6, or 8 franchises) and picks their franchise. The server automatically spawns `N - 1` deterministic bot franchises and loads the player dataset.	**Instant Auto-Start:** Because all `N` franchises are immediately occupied (1 Human + `N-1` Bots), the room transitions `CREATED` → `RUNNING` automatically and presents Player #1 after a 3-second countdown.
Mode 2: Play with Friends (Multiplayer)	A player creates a room, selects total team capacity `N` (or hybrid human + bot slots), and receives a shareable `roomCode`. Friends join via code and claim an available franchise.	**Capacity Auto-Start:** No manual "Start" button needed. The moment the `N` th franchise slot is claimed (or all players mark `READY`), the backend atomically flips the room to `RUNNING` and starts Lot #1.

2. System Architecture & End-to-End Automated Flow

SECTION 02

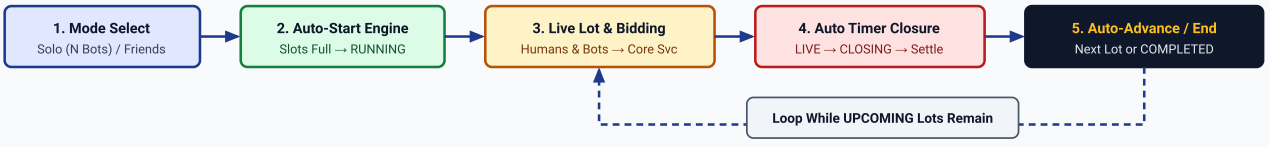


Figure 1: Fully Automated Auction Lifecycle — Zero Manual Host Intervention Required

How to Think About the Modular Monolith Architecture (Stack-Agnostic)

Do not couple your thinking to any single framework. Whether you code in Node.js, Go, Java, or Python with MongoDB or PostgreSQL, structure your backend as a **Modular Monolith** with four strict boundaries:

1. **Transport Layer (HTTP + WebSockets):** Authenticates caller identity, applies rate limits, accepts commands (`POST /bids`), and broadcasts committed state deltas (`bid:accepted` , `player:settled`).
2. **Core Auction Domain Engine:** The single brain. Validates bid increments, squad limits, and purse reserves. *Both human HTTP requests and automated Bot bids must call the exact same domain method.*
3. **Automated Lifecycle & Bot Worker:** A background loop driven by the server clock that auto-starts full rooms, evaluates deterministic bot bids, closes expired player timers, and auto-advances to the next player.

3. Domain Rules, Data Model & Authoritative Truth

SECTION 03

Reserve Purse Math & Provisional vs. Final Sale: Every franchise starts with an integer `startingPurse` (e.g., 10,000 units) and must build a squad between `minSquad` (e.g., 15) and `maxSquad` (e.g., 18) with at most `maxOverseas` (e.g., 6) overseas players. A rich franchise can still be **ineligible to bid** if: (a) its squad is full, (b) the active player is overseas and the team already has 6 overseas players, or (c) the bid violates its **Minimum Reserve Purse**—if a team has 11 players and `minSquad = 15`, it needs 3 more players *after* the current one, so it must keep `3 × minBasePrice` untouched. A leading bid is strictly *provisional*; money is deducted only when the timer expires and the sale is settled.

Entity	Key Authoritative Fields	Critical Unique Indexes & Invariants
AuctionRoom	`mode (BOTS|MULTI), teamCapacity, status, activePlayerId, rulesConfig, version`	Unique: `roomCode`. Auto-starts when `joinedCount == teamCapacity`.
Franchise	`roomId, userId (null if Bot), botArchetype, remainingPurse, totalPlayers, overseasCount`	Unique: `(roomId, name)`. Invariant: `remainingPurse >= 0` at all times.
RoomPlayer	`roomId, lotOrder, role, isOverseas, basePrice, status, currentBid, leadingFranchiseId, expiresAt, version`	Index: `(roomId, status, lotOrder)`. Only 1 player per room may be `LIVE` or `CLOSING` at once.
BidRecord	`roomId, playerId, franchiseId, amount, idempotencyKey, createdAt`	Unique: `(roomId, idempotencyKey)`. Immutable audit log of accepted bids.
ConfirmedPurchase	`roomId, playerId, franchiseId, finalPrice, settledAt`	**CRITICAL Unique:** `(roomId, playerId)`. Physically prevents double-selling a player.

4. State Machines & Automated Lifecycle Transitions

SECTION 04

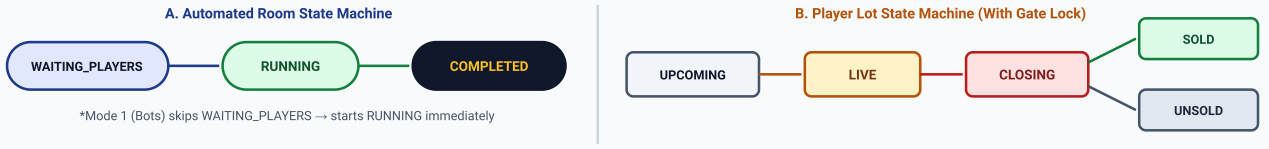


Figure 2: State Machines — The Intermediate "CLOSING" State Blocks Late Bids While Settlement Runs

Automated Workflow	How the Engine Executes It Safely Without a Human Host
1. Auto-Room Start	In Mode 1, room creation spawns `N-1` bots and starts immediately. In Mode 2, when the final friend joins (`joined == N`), an atomic conditional update transitions `WAITING_PLAYERS → RUNNING` and triggers `activateNextPlayer()`.
2. Bid Validation & Anti-Snipe	Checks `status == LIVE`, `serverNow < expiresAt`, valid increment, not self-bidding, squad caps, and purse reserve. If valid, updates `currentBid`, increments `version`, and extends `expiresAt = max(expiresAt, now + 8s)`.
3. Timer Expiry & Settlement	Background worker polls active lots where `serverNow >= expiresAt`. Atomically flips `LIVE → CLOSING`. Inside one DB transaction: inserts `ConfirmedPurchase`, deducts `Franchise.remainingPurse`, marks `SOLD` (or `UNSOLD` if no bids), and automatically calls `activateNextPlayer()` after a 3-second pause.
4. Deterministic Bot Loop	When `player:live` or `bid:accepted` fires, non-leading bots wait a random jitter (500–1200ms), compute a formula valuation (`basePrice × roleNeedWeight × budgetRatio`), and call the standard `submitBid()` service.

5. Crucial Concurrency Hazards, Deadlocks & Production Issues

SECTION 05

Why Single-Threaded Event Loops Still Have Race Conditions: Even in single-threaded runtimes like Node.js, database I/O is asynchronous. When two players click "Bid" 2 milliseconds apart, Request A and Request B *both* read the old price (`200`) from the database before either writes its update. If you use a naive `read → check in code → write` pattern, you will suffer lost updates, double sales, and negative budgets.

Production Issue / Race	Exact Failure Mechanism in Production	How to Solve & Architect Around It
1. Simultaneous Bid Collision (TOCTOU Race)	Team A and Team B both read `currentBid: 200, version: 3` at the same millisecond and both submit `220` .	**Optimistic Concurrency Filter:** Update query must include `WHERE _id = P1 AND status = 'LIVE' AND version = 3` and increment `version` to `4` . One succeeds; the other matches 0 rows and returns `409 Outbid` .
2. Bid Arrives at Exact Millisecond of Timer Expiry	A user's bid hits at `12:00:00.001` while the automated closure worker runs at `12:00:00.000` .	**Atomic Gate State (`CLOSING`):** Worker atomically updates `WHERE status = 'LIVE' SET status = 'CLOSING'` . Since bids also filter by `status = 'LIVE'` , the database serializes the two writes—whichever arrives second fails cleanly.
3. Duplicate Timer Workers / Double Settlement	Two timer ticks (or two server instances) pick up the same expired player and deduct the winning team's budget twice.	**Atomic Claim + Unique Index:** Only the worker whose `LIVE → CLOSING` transition returns `modifiedCount == 1` runs settlement. Additionally, a unique index on `ConfirmedPurchase(roomId, playerId)` blocks duplicate inserts.
4. Crash Mid-Settlement (Partial State Corruption)	Process crashes after marking player `SOLD` but before deducting `Franchise.remainingPurse` .	**Multi-Entity ACID Transaction:** Execute Player update, Purchase insert, and Franchise budget deduction inside one database transaction. On server reboot, a recovery scan re-settles any lot stuck in `CLOSING` .
5. Network Timeout & Duplicate Client Retries	Mobile Wi-Fi drops after sending a bid; client retries, causing the user to accidentally bid twice or error out.	**Idempotency Keys:** Client sends a UUID `Idempotency-Key` header per click. Server stores the key in `BidRecord` ; retries with the same key return the original response.
6. Split-Brain UI After Socket Disconnect	A player's WebSocket drops for 4 seconds, missing two bids and a player sale, leaving stale numbers on screen.	**Authoritative Reconnect Snapshot:** Never rely on WebSocket events alone. On socket `connect/reconnect` , the UI must call `GET /rooms/:id/snapshot` to overwrite local state and recalibrate clock offset.

Crucial Deep Dive: Database Deadlocks, Lock Contention & Connection Pool Starvation

When you write manual backend code with transactions and concurrent requests, you will encounter three classic production killers:

A. AB-BA Database Deadlocks: Suppose Transaction 1 (Auto-Settlement) locks `RoomPlayer P1` first, then attempts to lock `Franchise F1` to deduct budget. At the exact same moment, Transaction 2 (a concurrent franchise update or multi-entity check) locks `Franchise F1` first, then attempts to lock `RoomPlayer P1` . Transaction 1 waits for F1; Transaction 2 waits for P1. Both freeze in a **circular wait deadlock** until the database aborts one with a `WriteConflict / DeadlockDetected` error.

→ *How to Prevent It:* Enforce a strict, global **Lock Acquisition Hierarchy** in every function you write: always lock **1st: Room → 2nd: RoomPlayer → 3rd: Franchise → 4th: ConfirmedPurchase**. Never acquire locks in reverse order.

B. Hot-Document Lock Contention (Bot + Human Stampede): If 7 bots and 1 human all react within 5ms to `player:live` , all 8 requests will hammer the same `RoomPlayer` row simultaneously, causing massive

7. Core API & Real-Time Event Contract

SECTION 07

Endpoint / Socket Event	Input & Server-Side Authority Rule	Outcome & Real-Time Synchronization
`POST /api/rooms`	`{ mode: 'BOTS'|'MULTI', teamCount, purse }`. Server generates room & loads player pool.	If `mode == 'BOTS'`, spawns `N-1` bots, sets `RUNNING`, and auto-starts Lot #1.
`POST /api/rooms/:code/join`	Multiplayer friend claims a franchise slot.	When `joinedCount == teamCount`, server auto-transitions to `RUNNING` and emits `room:started`.
`POST /api/rooms/:id/bids`	Header: `Idempotency-Key`. Body: `{ playerId, amount }`. (`franchiseId` from auth token!).	`200 OK` → emits `bid:accepted`; `409 Conflict` if outbid/closed; `422` if rule/budget violated.
`GET /api/rooms/:id/snapshot`	Fetched on page load & socket reconnect.	Returns active player, server `now` (for timer sync), all franchise budgets, and squad lists.
WS: `player:live` & `player:settled`	Pushed by automated lifecycle worker *after* DB commit.	Updates client stage when timer closes (`SOLD/UNSOLD`) and when next player auto-starts.

8. Manual Debugging, Concurrency Stress Testing & 4-Week Plan

SECTION 08

How to Debug & Prove Your Code Works Manually: Do not rely on clicking buttons in one browser window. First, add a **Correlation Request ID** and log every state transition (`[reqId] Player P1 version 3->4 by TeamB`) so you can trace interleaved requests in your terminal. Second, write a 25-line script that fires **50 simultaneous requests** using `Promise.all()` (or parallel threads) against your local server:

- **Experiment 1 (The 50-Bid Race Storm):** Have 6 franchises fire 50 simultaneous bids for the exact same next increment (`220`) at the same millisecond. Verify that **exactly 1 succeeds (`200`)**, 49 fail with `409 Conflict`, and `version` increases by 1.
- **Experiment 2 (The Boundary Deadline Race):** Fire 10 bids within 5ms of `expiresAt` while the automated closure worker runs. Verify that either a bid extends the timer *before* `CLOSING`, or it is rejected—never a silent overwrite after `SOLD`.
- **Experiment 3 (The Accounting Invariant Check):** After an automated 30-player auction completes, run an assertion script verifying that no player has >1 `ConfirmedPurchase` and every franchise satisfies `remainingPurse + SUM(purchases) == startingPurse`.

Realistic 4-Week Manual Implementation Roadmap (90 Mins/Day Alongside Exams & DSA)

• **Week 1 (Domain & Persistence):** Design schemas & indexes; manually code Room creation (Mode 1 & Mode 2), Franchise joining, automated start trigger, and sequential player activation.
• **Week 2 (Core Bidding & Atomic Settlement):** Code increment slabs, reserve purse math, squad limits, the automated deadline worker (`LIVE → CLOSING → SOLD/UNSOLD`), and multi-entity ACID settlement transactions.
• **Week 3 (Concurrency Hardening & Real-Time Sync):** Add optimistic `version` filters, `Idempotency-Key` handling, WebSocket broadcasts, reconnect snapshots, and pass the 50-request concurrency stress script.
• **Week 4 (Deterministic Bots & Frontend Polish):** Add the deterministic bot evaluator loop for Mode 1, build the clean Live Auction & Squad UI screens, and test full automated game loops.

9. Using AI Strictly as a Design Critic (Zero Code Generation)

SECTION 09

```
// Copy-Paste This Prompt When You Want AI to Critique Your Manual Design or Help You Debug an
Error:
"I am manually coding AuctionArena to learn backend engineering. DO NOT write or generate code for
me.
Here is my design / error for [Automated Timer Closure / Bid Validation / Multiplayer Auto-Start]:
```